



Figure 1: The KVM process model

slightly different.

The new extensions add a new mode, 'guest mode', in addition to the user-mode and kernel-mode that we had (ring -1 in addition to the rings 0-3, with the hypervisor residing in ring -1). The implementations also enable support for hiding the privileged state. Disabling interrupts while in guest mode will not affect the host-side interrupts in any way.

The KVM way

For KVM, the focus is on virtualisation. Scheduling of processes, memory management, IO management are all pieces of the puzzle that were already available. It didn't make sense writing all those components again, especially since the software available was in wide use, was very well tested (hence stable), and written by experts in the field.

The way KVM solves the virtualisation problem is by turning the Linux kernel into a hypervisor. Scheduling of processes and memory management is left to Linux. Handling IO is left to *qemu*, which can run guests in userspace and has a good device model. A small Linux kernel module has been written and it introduces the 'guest' mode, sets up page tables for the guest and emulates some instructions.

This fits nicely into the UNIX mindset of doing one thing and doing it right. So the *kvm* module is all about enabling the guest mode and handling virtualised access to registers. From a user's perspective, there's almost no difference in running a VM with KVM disabled and running a VM with KVM enabled, except, of course, the significant speed difference (Figure 1).

The philosophy also extends to the development and release philosophy Linux is built on: release early and often. This allows for fast-paced development and stabilisation. Developers can track the latest and greatest codebase and keep enhancing it.

The latest stable release is part of Linux 2.6.x, with bug fixes going in 2.6.x.y. The KVM source code is maintained in a git tree. To get the latest KVM release or the latest git tree, head to kvm.qumranet.com for download details.

Using KVM

First, since KVM only exploits the recent hardware advances, you should make sure you have a processor that supports the virtualisation extensions. This can be quickly found out by using the following command:

```
egrep --flags="(vmx|svm)" /proc/cpuinfo
```

If there's any output, it means the necessary capability to run KVM exists on the CPU. If you have a CPU that's less than three years old, you're set.

You now need to run a recent 2.6 Linux kernel. If you already run a recent Linux kernel with KVM either compiled in the kernel or compiled as modules, you can work with it if you don't want to compile the modules yourself. All the distributions these days ship *kvm* modules by default, so getting KVM is just a matter of fetching the necessary packages from your distribution's site if they're not already installed.

There are two parts to KVM: the kernel modules and the userspace support, which is a slightly modified version of *qemu*.

You can also download the KVM sources from *kvm.qumranet.com/kvmwiki/Downloads*.

Building the userspace utilities from the tarball needs to have a few libraries present. The detailed list and instructions are given at kvm.qumranet.com/kvmwiki/HOWTO.

Once you have the kernel module and the userspace tools installed (either by building or installing from your distribution packages), the first thing to do is create a file that will hold the guest OS. We have previously seen how to do that using the virtual-manager GUI in January. If you like using the command prompt, which is my preferred way of doing it, here's a quick tutorial:

```
$ qemu-img create -f qcow debian-lennyimg 10G
```

This will create a file called *debian-lenny.img* of size 10G in the 'qcow2' format. There are a few other file formats supported, each with their advantages and disadvantages. The *qemu* documentation has details.

Once an image file is created, we're ready to install a guest OS within it. First, insert the *kvm* kernel modules in the kernel if they have been compiled as modules.

```
$ sudo modprobe kvm
$ sudo modprobe kvm-intel
```

...or:

```
$ sudo modprobe kvm-amd
```

On Debian-based systems you can add yourself to